

O'REILLY®

Early
Release

RAW &
UNEDITED

Compliments of
 Antithesis



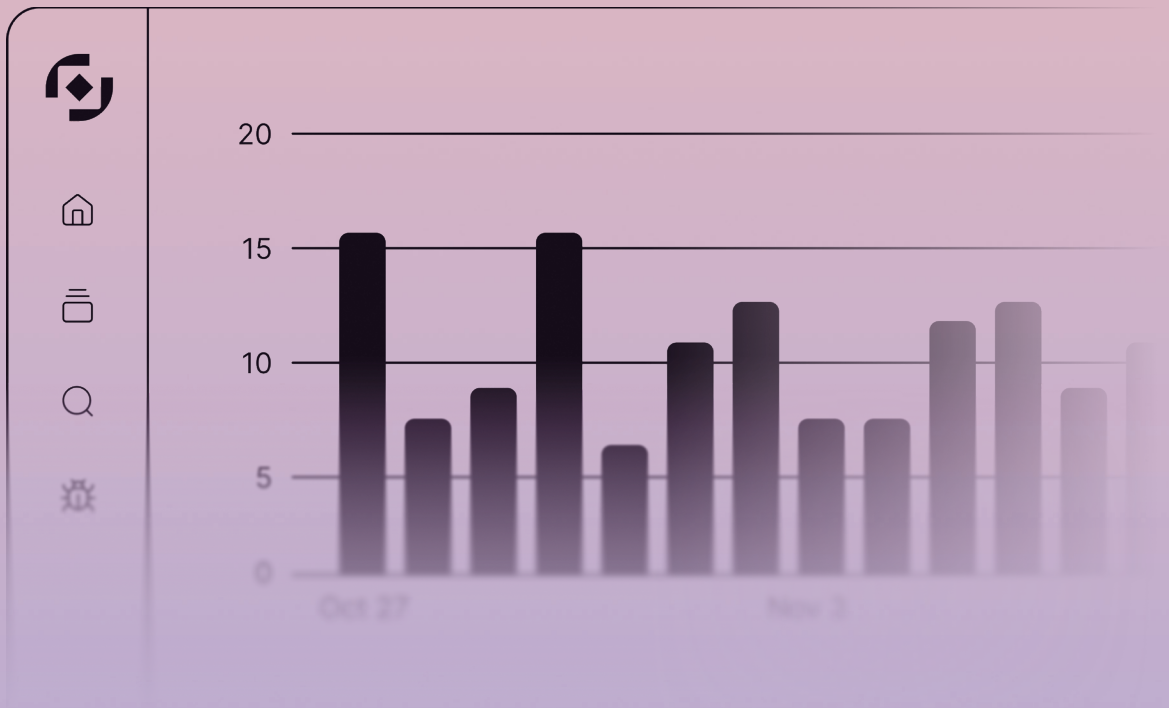
Scaling AI Adoption in Engineering

A Practical Framework for Leaders

Peter Bell

■ RELEASE WITH CONFIDENCE

Verify your code in CI, dev, & /loop.



Learn more at
antithesis.com

 Antithesis

Scaling AI Adoption in Engineering

A Practical Framework for Leaders

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Peter Bell

O'REILLY®

Scaling AI Adoption in Engineering

by Peter Bell

Copyright © 2026 Gather.dev, Inc. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: David Michelson

Development Editor: Sarah Grey

Production Editor: Elizabeth Faerm

Copyeditor: TO COME

Proofreader: TO COME

Indexer: TO COME

Cover Designer: Susan Brown

Cover Illustrator: TO COME

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

July 2026: First Edition

Revision History for the Early Release

2025-11-17: First Release

2026-01-20: Second Release

2026-05-13: Third Release

See <https://oreilly.com/catalog/errata.csp?isbn=9798341666856> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Scaling AI Adoption in Engineering*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-66681-8

[LSI]

Table of Contents

Brief Table of Contents (<i>Not Yet Final</i>).....	v
Preface.....	vii
1. A Tale of Two Orgs.....	1
Flatline Systems	1
Summit SaaS	3
Where is your org?	5
2. AI as an Amplifier.....	7
Culture and Trust	7
Customer-Centered Collaboration	8
Engineering Excellence	9
Developer Experience and Enablement	10
Change, not tools	10
Summary	12
3. What's Your Strategy?.....	13
Do you AI?	14
Building Competence	14
Build a research agent	14
Drive the process	15
Reviewing your strategy	15
Impact Assessment	15
Understand your audiences	17
Customers and Prospects	17
Employees and New Hires	18
Investors	18

Assess your engineering maturity	19
Picking a Lane	19
Innovator	19
Early Adopter	20
Early Majority	21
Late Majority/Laggards	22
4. What's Your Plan?.....	23
Aligning Across the Org	23
Your talent team	24
Your Engineering org	24
Planning Implications by Adoption Strategy	24
Innovator	24
Early Adopter	27
Early Majority	29
Late Majority/Laggard	30

Brief Table of Contents (*Not Yet Final*)

Preface (available)

Part 1: Setting the Context (unavailable)

Chapter 1: A Tale of Two Orgs (available)

Chapter 2: AI as an Amplifier (available)

Chapter 3: What's Your Strategy? (available)

Chapter 4: Building Personal Competence (unavailable)

Part 2: The Agentic SDLC (unavailable)

Chapter 5: A Glimpse Into Our Agentic Future (unavailable)

Chapter 6: From Single to Multi-Player Mode (unavailable)

Chapter 7: How to Build a Software Factory (unavailable)

Chapter 8: Capturing and Delivering Context (unavailable)

Chapter 9: Compounding and Dreaming (unavailable)

Chapter 10: Verification (unavailable)

Chapter 11: Deploying and Operating Software (unavailable)

Part 3: The Agentic Engineering Org (unavailable)

Chapter 12: Structuring the Engineering Org (unavailable)

Chapter 13: Hiring and Onboarding (unavailable)

Chapter 14: Managing AI Risk, Security, and Governance (unavailable)

Chapter 15: Managing Teams (unavailable)

Chapter 16: Managing Projects (unavailable)

Chapter 17: Managing Tokens (unavailable)
Part 4: Leading the Org Through the Transition (unavailable)
Chapter 18: Performing a Readiness Assessment (unavailable)
Chapter 19: Managing Expectations (unavailable)
Chapter 20: Tool Selection and Approval (unavailable)
Chapter 21: Driving Adoption (unavailable)
Chapter 22: Leading in Uncertain Times (unavailable)
Epilogue: Do CTOs Have a Future? (unavailable)

Preface

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the Preface of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

If you’re a CTO, I have good and bad news for you. The bad news is that many organizations are getting *no* ROI from adopting AI within their engineering orgs (according to the 2025 **NANDA** and **METR** studies). The good news is that other companies are seeing a *tremendous* acceleration in their ability to deliver value. The best news is that the differences between the high- and low-performing orgs are now becoming clearer—and so is how to move from one category to the other.

If you are a software engineering leader (especially if you’re running an org of 100+) and you’re looking for good practices for adopting and scaling AI to accelerate how your org delivers business value, this book was written for you. The synthesis of hundreds of interviews with “CTOs at Scale” and executives from top AI companies, it is designed to provide you with a comprehensive plan for experimenting with and scaling the adoption of GenAI across your org. Recognizing that the hardest parts of developing software effectively are almost always about humans and very seldom about code, it focuses on the strategies, stakeholder engagement, and change management you’ll need to lead your org through turbulent times.

Here are some things this book is *not* focused on:

How LLMs work

I'll assume you already know enough to be able to adopt them.

Vibe coding

I have a chapter on augmented (IDE-based) and agentic coding, but the hard part of adopting LLMs is scaling value across the org, not just improving individual efficiency.

Architecting GenAI systems

I'll share some good practices on creating a shared internal AI infrastructure to improve quality, observability, and control, but this isn't a book on architecting AI products.

Operationalizing inference

I'll share some hard-won wisdom about hosting internal GenAI tooling, but this is not primarily a book about MLOps.

Specific GenAI tools or infrastructure

I may mention some examples of tools, and I'm busy building a **whole catalog of AI dev tools**, but specific recommendations change too quickly to commit to paper.

AI enabling your business offerings

This book is focused on transforming your engineering org. Repositioning your company is a wider conversation that I will only briefly touch on, in Part 1.

AI-enabled business processes

There are substantial efficiencies to be gained by leveraging AI across all business functions, but this book is focused on the subset of processes that directly support the software development lifecycle (SDLC).

The broader impact of AI on society

There is a huge cone of uncertainty as to AI's long-term impact on society, jobs, public discourse, inequality, and—potentially—the future of humanity. I'd be delighted to discuss any or all of those over a drink.

This book *is* focused on the more tractable and immediate goal of helping CTOs successfully drive the adoption of AI within the SDLC. It's aimed at helping engineering leaders navigate what might be the biggest organizational change of their careers.

Using this book

Start with Part 1. It's a quick read and it's designed to provide context, without which the rest of the book would be tactics without a unifying theory or strategy.

In **Chapter 1**, I tell a “tale of two orgs,” synthesizing hundreds of interviews into a vignette of two fictitious companies (Flatline Systems and Summit SaaS) that epitomize the two extremes between which most real companies fall when adopting AI for engineering. I then touch on some key takeaways that I’ll unpack in more detail throughout the rest of the book. In **Chapter 2**, I expand on the concept of AI as an amplifier, highlighting some of the organizational traits that maximize the value available by adopting LLMs.

In Chapter 3, I take a step back so we can examine your broader business context. Most engineering orgs can get value from LLMs, but some face a more urgent, existential threat from AI than others. If you’re selling website-building SaaS software, you’d better be one of the innovators in adopting AI. If you manage ski resorts, the risk of GenAI is less existential—whatever happens with LLMs, people are probably still going to be skiing 10 years from now. I provide a framework for thinking about AI adoption and “picking a lane” for your company, whether that’s as an innovator or as a more cautious “late majority” adopter.

In Chapter 4, now that we have clarity on how aggressively your organization wants to adopt GenAI for software development, I share some good practices for aligning with the security, audit, risk and legal teams to ensure you are capable of approving experiments and unblocking your team.

With that framework in place, in Part 2 I take some time to envision what engineering orgs might look like in the future. I share what’s working now and possible futures for the entire SDLC, from specification through coding, reviewing, testing, and deploying new features. I also look at what’s working for all of the supporting activities that go on within engineering orgs, from management to hiring to onboarding.

Finally, in Part 3, I provide detailed guidance on the practical steps required to build out an AI-powered engineering org effectively: from executive alignment, managing stakeholder expectations, and setting employee expectations to building personal competence, staffing the initiative, partnering with product, tool selection and approval, identifying experiments, driving adoption, and leading in uncertain times.

A Tale of Two Orgs

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

Some companies are getting transformative results from large language models (LLMs); others are just wasting their money and frustrating their engineers. That’s the *GenAI divide*. In this chapter, to highlight what’s at stake, I’ve synthesized dozens of conversations with CTOs into two composite stories so we can compare and contrast two approaches to engaging with AI in the software development lifecycle (SDLC). I then pull out some high-level insights from the CTOs who are getting the best results from their AI transformations.

There is no element of these composites that I’ve not encountered in the wild.

Flatline Systems

Flatline Systems is a publicly traded AdTech company with over 800 engineers, a third of whom are contractors from a large outsourcing firm. Flatline builds low-latency, high-performance ad servers for connecting publishers to advertising opportunities in real time. Its code is mainly a collection of Java microservices, with C++

for much of the ad-server code. It has most of its apps running on public clouds (with just a few on-prem holdouts). Test coverage is spotty, manual QA is required for every release, and deployment of many of the apps still isn't automated.

The CTO at Flatline knew he needed to show decisive action on GenAI. As soon as it was announced, he inked a deal to provide Enterprise Copilot access to an initial test group of 20 engineers to see how much value it could deliver.

The CTO was already overwhelmed with his existing roadmap, but he asked one of his newest directors to shepherd the pilot and to report back each quarter. He couldn't pull the director off of her main project, which was mission-critical, but he added a sixth quarterly goal to make sure she'd be motivated to make the pilot a success.

At first, the director didn't seem to understand the assignment. She wanted to completely restructure the pilot, pick teams rather than individuals, and go through a complicated opportunity-identification process. It made sense, but the CTO had already provisioned the licenses and had more urgent priorities.

The director kept on asking for more support. She wanted access to Claude and Cursor, even though she already had the Copilot licenses. She wanted to trial additional niche tools for test generation, code review, and DevOps automation. Most of the companies she wanted to work with were early-stage venture-backed companies with very little track record. It wasn't even worth taking it to the legal team or the CISO—they'd never sign off on it.

She asked for a budget for onsite training, found some great trainers, and obtained quotes, but it was really hard to justify spending tens of thousands of dollars on training when the internet is full of blog posts and videos on how to use LLMs.

She also asked if the engineers could get extra time to deliver their projects. It really made no sense. The whole purpose of the experiment was to speed up the dev team, not to slow them down.

The worst part was that while some of the engineers seemed to enjoy using Copilot, according to their managers, there was no obvious improvement in velocity or code quality. This result was subjective, of course, because it's really hard to measure developer productivity and the CTO could never justify the cost of implementing developer-productivity tools. However, it seemed consistent with reports suggesting that many companies get no ROI from adopting LLMs.

He didn't fire the director for messing up the pilot, but he made it clear that she was on probation and had better impress him with her other initiatives if she wanted a satisfactory rating. She ended up leaving the company shortly afterwards. Clearly he'd made another poor hiring decision.

The CTO didn't want to be seen as behind the curve, so despite the lack of proven value, he decided to allow engineers to request a Copilot license—as long as they could commit to at least a 10% productivity increase (to be evaluated by their team lead). Strangely, nobody seemed to care enough about AI to sign up for the licenses. In addition, when he looked at his regrettable attrition statistics later in the year, he noticed that a disproportionately high percentage of engineers from the original AI pilot had left the company. Another downside not enough people were talking about!

The CTO was always a little nervous when new case studies came out showing dramatic improvements at other orgs that were leveraging LLMs. He asked a couple of distinguished engineers he respected what was up. They explained that while LLMs are great for startups, they just don't work at enterprise scale and with legacy code, especially in niche domains: “The training data just isn't there. Autocomplete is never going to replace experienced engineers solving hard problems.”

The CTO still felt like he might be missing something, but the data seemed clear, and he really needed to focus on his Q4 deliverables to support two big new partnerships the sales org had brought in. He figured he could always check in again in six months to see if it was worth revisiting his AI strategy.

Summit SaaS

The engineering org at Summit SaaS had a lot going for it even before it started to experiment with LLMs. It was a series C, venture-backed dev tool company that seemed to attract the best engineers, both at its San Francisco headquarters and across the US and Europe. They had a motivated, remote-first org of 180 engineers. Summit paid 85% of top of market in San Francisco, and compensation was independent of geography, so they were able to get some particularly capable engineers outside of the Bay Area.

Summit had a mature platform team of 12 responsible for delivering the tooling required for the feature teams to ship faster. It was one of the first companies to rename its platform team a “DevEx” team, to reflect the team's commitment to optimizing the developer experience for their customers (the engineers).

The DevEx team members weren't religious about Test-Driven Development, but they had about 90% code coverage, ran continuous integration (CI) on all branches, used red/green automated continuous deployment (CD), and leveraged a mature feature-flagging system for running experiments in production. They also had excellent observability, monitoring, and notifications for system, application, and business metrics.

The CTO believed deeply in the potential of LLMs and was an early adopter of LLMs personally, often creating her own prompts to help with communications and planning. She created a lightweight set of AI agents for ingesting data from systems of

record, powered by Markdown files, n8n, and a couple of Model Context Protocol (MCP) servers. This allowed her to automate a lot of the reporting that the data team hadn't had the time to build out yet.

The CTO's DevEx team had a substantial roadmap, but she believed deeply that GenAI was an investment worth making. After aligning with the board and her CEO, she asked her Director of DevEx (who ran the platform team) to make this his primary focus for the next 18 months, backfilling his role with a senior engineering manager to lead the rest of the platform org.

The director was also passionate about LLMs and had already been using them in his daily work. He started by agreeing with the CIO, the COO, and their General Counsel on some "red lines" for AI adoption: things they would not do, like allowing third-party cloud-hosted models to train on their company data. Then, to get some hands-on experience, he vibe-coded a lightweight internal tool for managing AI pilot requests across the company. The executive team committed to an SLA of three business days for reviewing and approving or (transparently) denying requests. There were no ROI requirements for experiments costing under \$10k a month in licensing fees; engineers just had to avoid the red lines to manage critical business risk.

They assembled a cross-functional AI adoption team that met weekly to share what they were learning. Within engineering, the director created a simple system to allow anyone to propose potential use cases for intensive AI experimentation, with a clear and transparent rubric for prioritizing those larger investments. He also hired one full-time technical project manager (PM)/trainer, a former developer relations engineer, to manage the logistics of the pilots, track what they learned, unblock the teams, and organize regular comms and activities to help the team to share what was working.

They also provided their entire engineering org with access to multiple general-purpose coding tools for both augmented (IDE-based) and agentic engineering. Over the next 18 months, they also ran experiments with niche tools for performing code and security reviews, automating DevOps requests, modernizing some of their older apps, and generating tests and documentation. They experimented with multiple tools in each category and allowed different teams to run experiments in parallel.

The CTO made it clear that they were going to be an AI-forward engineering org, and that a 10 to 15% decline in short-term productivity was acceptable—providing that it accelerated AI learning and adoption across the engineering org. During Summit's annual hack week, she asked everyone to focus on leveraging AI in their submissions—including the business ops teams, who vibe-coded some amazing prototypes for backend reporting and some customer-requested features. She also encouraged her teams to take a little slack time to play with LLMs more intensely.

She spoke frequently at conferences about AI adoption and retooled her hiring process to *require* engineers to use LLMs in the interview process. She wanted to attract the engineers who were most excited about adopting AI in their work.

A number of the experimental projects failed, and she noticed slightly higher levels of regrettable attrition than usual. In exit interviews, the engineers often said that they didn't want to become "engineering managers to a bunch of agents." But her AI adoption team coalesced around a few successful tools that they rolled out across the org. They also started to build out a company-wide context repository to hold everything: business goals, go-to-market strategies, customer-service conversations, engineering experiments, and automatically generated documentation for the app's capabilities.

In partnership with the product org, the team also started to experiment with agents for streamlining the process of going from objectives and key results (OKRs) to experiments to product requirements documents (PRDs) to tickets to code. Often the agents would get carried away, but over time they found that 30 to 40% of their app's features could be coded agentically, with a human reviewing and cleaning up that code. The rest could be accelerated by 30 to 80% by leveraging augmented engineering within an IDE. They also started to get much more rigorous about the format of PRDs and started committing them to a GitHub repo. They noticed a direct correlation between the specificity of their PRDs and the quality of the resulting agentically generated code.

They also experimented with using LLMs to track engineer engagement, support 1:1 meetings, bring more data to performance reviews, and automate a lot of the project management across the company. They still had a great Technical Project Manager, but her role evolved from administrative to consultative, allowing her to focus much more on alignment and prioritization and much less on reporting and documentation.

The CTO was clear that "LLMs right now are as bad as they are ever going to be" and that they still had a long way to go in learning how to use them effectively. But with some clear wins, broad adoption, and a team excited to leverage the latest tooling, she was excited to continue trialing the latest tools, with the goal of maximizing her team's ability to deliver business value in a relaxed but productive manner.

Where is your org?

Most companies' AI adoption is somewhere in between Flatline Systems and Summit SaaS, but the two extremes help to highlight some key commonalities in companies that are getting the best ROI from their GenAI initiatives.

AI isn't something you can just read about or delegate. If you're not personally using AI daily, you won't be able to lead your org successfully through your AI transition. You should be experimenting with LLM-based tools for presentations (like

gamma.app), building lightweight context about your org in Markdown files, and running Claude Code subagents to automate regular reporting. You should find a little time to futz with MCP servers, and you should start to think about how you'd need to persist and flow data within your org to improve productivity not just by 10 to 30%, but 10x.

AI requires investment. Change management is a full-time job that requires funding and support, and your team may be a little less productive as they build the skills required to leverage AI effectively. Give them time to experiment, while still requiring that the code they choose to commit must be clean and well architected, no matter how they create it.

You're going to need an executive with platform/DevOps experience within the engineering org who is focused on driving your AI experimentation and adoption. They should prioritize enabling an efficient flow of work from specification to production, and they should have enough experience and credibility within the org that they'll be able to get *everyone*—from individual contributors to executives (think the CISO and the audit and legal teams)—to engage with their proposals.

You also need to think about incentives. Saying “I need a proven ROI from every token used” creates very different incentives than saying “I’ll promote the people who are burning the most tokens.” In the early experimentation phase, the latter will accelerate your learning—even though not every experiment will prove fruitful.

You need to get out of your team's way. If you don't allow them to experiment with all the tools that are safe to try, they'll either hide their usage (“shadow AI”) or move to a company that allows them to run experiments more quickly and with less bureaucracy.

You need to rethink hiring, too. Using AI isn't cheating: it's the job. As a CTO, the way you position AI use in the hiring process sends a very clear signal about the kind of company you want to be. Do you want to attract or deter engineers who are excited to augment their capabilities with LLMs?

Most importantly, you need to focus on the fundamentals. It turns out that AI is an amplifier; the more efficiently your engineering org operates *without* LLMs, the more value they will get from adopting them.

AI as an Amplifier

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

The teams that are getting the greatest value out of AI are the ones that already have the strongest culture, the clearest customer focus, the most robust engineering practices, and the slickest developer experience. In this chapter I highlight some of the consistent elements I’ve seen in the companies getting the best ROI from adopting LLMs.

In Part 3 of the book, “Building the AI-Powered Engineering Org,” I’ll dig a lot more deeply into identifying your organizational maturity in each of these areas and improving your processes and practices. Think of this as a high-level cheat sheet for reviewing how ready your organization is for AI-based acceleration.

Culture and Trust

AI adoption is much easier when you have a trusting relationship with your team. Change is scary—especially when it’s not your idea. AI is likely to transform the composition and nature of roles across product and engineering. Without a supportive culture, your team is unlikely to take the risks necessary to succeed with AI.

None of us knows exactly how AI is going to impact the nature of programming jobs. You may not be able to make promises about what the future will hold. But if your team knows that they can trust you to be honest and transparent and that you care about their wellbeing and careers, they are much more likely to feel safe when experimenting with new ways of working.

The nature of experiments is that many of them don't work out. We're still learning the right way to leverage LLMs to accelerate the SDLC. It's important to decouple the success of experiments from performance reviews, bonuses, and career development. If you don't, nobody will be willing to risk taking on experimental projects.

While it's important that you provide a strong mandate for adopting AI, you also need to work hard to solicit tactical advice and experience from the people who are actually using the tools. Good practices for leveraging AI are not intuitive and are most often discovered through hands-on experimentation. It's the individual contributors who are most likely to identify how to get the most value out of LLMs. Make it safe for them to tell you when your plans don't make sense, so you can course-correct quickly.

AI is moving so fast that keeping up with it requires a lot of passion and commitment. I'm not talking about a "996" culture, where you ask your team to work 12 hours a day (9am to 9pm), 6 days a week. But when they are at work, they need to bring their curiosity, their love of learning, and a commitment to continual experimentation and improvement.

The value that LLMs can provide is directly correlated to the comprehensiveness and accuracy of the context available to them. More than ever, it's becoming critical to avoid relying on "water-cooler conversations" or even emails and private DMs for communication, and to instead use transparent, persistent communication mechanisms like wikis and public Slack channels for any information that is not fundamentally sensitive. It's the only way that *all* of your teammates—human and agentic—can know what's going on.

Customer-Centered Collaboration

The goal of the engineering org is to solve customer problems creatively. Classic Lean and Agile patterns are incredibly important when it comes to leveraging LLMs to that end.

Increasingly, the value of an engineer will come from their ability to specify solutions to business problems that are rigorous, architecturally sound, and maintainable. There will be far fewer "code monkey" roles, where engineers are simply tasked with implementing a spec and don't need to care about the business context.

You'll need small, self-sufficient teams capable of specifying, designing, developing, testing, and deploying features to production. If you don't have those, any improvements in coding time will be dwarfed by the queueing time incurred by coordinating schedules across multiple teams.

The smaller your pull requests and the less context they require, the less likely it is that your LLMs will start hallucinating API endpoints and functions that don't exist. Small, customer-valued slices of work allow for easier review, greater velocity, and quicker learning as you see which features actually move the needle on your business goals.

Engineering Excellence

The more rigorous your engineering practices are, the more you'll be able to trust your LLMs to generate code.

The great thing about Test-Driven Development is that it's a design methodology, not just a testing strategy. It forces you to think through even the simplest possible behavioral changes you want to make—and to define success before you start implementing those changes. That is also a great pattern to use with LLMs to make sure you're aligned on the outcomes before they start to work on an implementation. While it doesn't need to be used for all new features, TDD can be a great methodology for pairing with an LLM. You and the LLM agree on exactly what behaviors you want to provide and then you validate that it's delivering them.

LLMs will make mistakes (as will humans). But if you provide them a quick, automated method to validate success (that is, running a test suite), they're much more likely to be able to self-correct without intervention. It's also much easier to catch regressions when you have good test coverage and an easy way to fire off alerts and report on any errors encountered.

LLMs can accelerate the delivery of business value. Couple that speed with a focus on small, thinly sliced user stories, and it's likely that, as you adopt AI, you will be deploying to production much more frequently. It's important to automate your CD pipeline to keep up with the accelerating pace of feature delivery.

Everyone is testing in production, too, whether it's testing with real-world load or testing the edge cases that only come up with real users and real data. It's important to have robust feature-flagging tooling to decouple deployment from release. This allows you to roll out new features to users incrementally to ensure that they're working as expected.

It's always been important to have system, application, and business-level reporting and notifications, so that if a CPU is spiking, an application is timing out, or your number of active users drops by 50% in a minute, your team gets notified and can

resolve the issue quickly. With a faster pace of change and a higher potential for errors, it's easier and safer to go fast with LLMs when you have the guardrails in place to identify and notify you of problematic deltas in your telemetry.

Architecting for resilience is another great strategy. Futzing, game days, chaos engineering: there are lots of ways of describing the pattern, but at its heart it's about managing the blast radius of failures. If your payment provider goes down, can people still add to their carts? If your shopping cart is broken, can they at least view the products? While architecting resilience into your applications isn't essential to getting the best out of LLMs, it does provide defense in depth as you introduce one more variable that could break your applications.

Developer Experience and Enablement

It's critical to be thoughtful about the developer experience, both individually and in aggregate. Companies with a strong DevEx function are much more likely to be able to support their engineers in adopting LLMs successfully.

The great thing about having a platform team is that it gives you an obvious leverage point for driving your LLM initiatives within engineering. The platform team is already in the business of accelerating your feature teams' work, so LLMs just become one more tool they can use to help their customers (your feature teams) go faster.

Having a DevEx team is about more than just rebranding your platform org. It's a clear signal that the job of a platform team is to facilitate the feature teams' *experience* of developing software. The engineers are the platform team's customers, and maintaining a customer focus really helps the platform team to be thoughtful when proposing, implementing, and maintaining tooling for developers—whether that's a new internal developer portal (IDP) or a set of agentic coding tools.

Measurement is another key element of engineering success. If you want to improve your ability to deliver business value quickly without burning out your team, you'll need to invest in engineering efficiency metrics so you have a baseline for comparison—whether that's DORA, SPACE, DX Core 4, or your own unique mix.

AI is not a silver bullet or a quick win, but it can both create an impetus for change *and* accelerate the process of adopting new processes and practices.

Change, not tools

If you take just one thing away from this book, it should be that AI adoption is a change-management problem, not a tooling-selection problem. AI is going to fundamentally change the nature of your engineering organization. It's not like moving to the cloud or adopting a developer portal—in fact, the closest analogy might be managing through an acquisition.

You're getting acquired!

When a company is acquired, both the tech stack and the engineering org are audited and lots of hard decisions need to be made. Is this the right structure? Are we duplicating functions? Who do we keep in the same role and who do we promote, move, or let go?

To say the quiet part out loud: *we're going to have to make all of those decisions over the next few years as AI transforms the SDLC.*

Some of this is simply a continuation of trends that were already in play. Ever since Meta's Mark Zuckerberg introduced us to "the year of efficiency" in 2023, engineering orgs have seen a thinning of middle management, a focus on "player coaches," an expanding span of control, and higher performance demands on everyone.

There is still a lot of uncertainty and it's too soon to be making big moves, but your team knows that eventually, if LLMs fulfill their promise of greater efficiency, you're going to have to choose between an accelerated roadmap or a lower headcount. It's not obvious which you'll end up going with.

Creating a change-management strategy

AI tooling is new, but change management has been around for a long time, and there are classic techniques that can help you to steady and support your org and create an environment where people can share and adopt good practices effectively.

A commitment to change

You need to make it clear that you are already committed to this transformation. The goal of starting with experiments is to identify *how* to proceed with LLMs, not *whether* to do so.

Transparent communications

Be honest that you don't know how this will impact the size or the structure of the org, but that you're committed to helping your team to build the skills that will allow them to thrive as augmented engineers, at your company or (eventually) somewhere else.

Identify and empower champions

Empower members of your team to experiment with AI tooling and support them in doing so. Make sure you have someone responsible for quickly unblocking any obstacles to running your AI experiments.

Listen to the team

As part of your commitment to psychological safety, make sure there are channels where the engineers can be honest about what is and isn't working with their experiments, so you can understand what's happening and course-correct quickly.

Celebrate learnings

Make it safe to experiment by celebrating what you’re learning across the org—even if what you learned was that a particular approach isn’t effective for your use case!

Document progress

Get a member of your platform team to create a tool that synthesizes and summarizes your AI experiments and learnings. This can provide a library and a state of play and generate regular updates for those within and outside of the engineering org.

Consistent rituals

Schedule weekly AI demos, monthly roundups, and regular office hours with domain experts to support your team in figuring out how to leverage LLMs.

Structured training

Hold lunch-and-learns, group classes, and one-on-one mentoring sessions. For an org of 100 or more, you will need at least one full-time trainer focused solely on the logistics of creating and structuring resources and experiences to help your whole team get better at managing AI context and efficiently decomposing tasks.

Regular communications

Keep beating the drum: regular communications should be top of mind for everyone. At the very least, share a weekly progress summary. Take the time to include a short personal experience, experiment, or insight each week, so you’re setting a tone of continual learning and engagement with AI. Just block half an hour on your calendar every Friday morning right now, so you always have the time to ship it each week.

Summary

AI is an amplifier. The higher performing your engineering org is, the more benefit you will gain from leveraging LLMs. AI adoption is primarily about change management, not tool selection. AI adoption is not free: it’s an investment that may take time to pay off.

I’ve touched on a lot of topics here, and I’ll spend most of Part 3 of the book providing a lot more advice and information on how to effectively enable AI in your org.

But before you start enabling, you need strategic alignment. Chapter 3 looks at how you can review and improve your company’s AI plans to ensure they are consistent and appropriate for the business you’re in and the team and the executive mandate you have.

What's Your Strategy?

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

It's frustrating. I get it. Your CEO just forwarded another link to an article showing that 80% of code can be generated and that you should either fire most of your team or be delivering against the roadmap at breakneck speed.

The CFO emailed you a link to a company offering to generate all of your tests and asked why you still need to pay for a QA team. They also sent you a link to a company that provides DevOps agents and asked if you really need so much budget for headcount in platform next year. Then your CMO texted you an interview with the CEO of a competitor announcing their new AI product and asked what it would take for you to ship something better.

Take a breath. Maybe another one. OK. The bad news is, this is incredibly frustrating. The good news is that if you're the CTO (or SVP of engineering, if software development doesn't get a seat in the C-suite in your org), they hired you specifically to provide context and make sense of technology to help the business make the right decisions.

This is literally your job.

One of the most influential things a CTO can do right now is to help the senior leadership team and the board think through the implications and opportunities of AI as they relate to your organization.

While the rest of this book is specifically about adopting AI within the SDLC, for this chapter, we're going to take a broader perspective, looking at how you can bolster your executive credibility, the audiences you need to consider when reviewing your AI strategy, and the most important decision your executive team needs to make.

This chapter is particularly relevant to CTOs. However, even if you're not a CTO, you should still give it a skim—it'll help you evaluate the implications of your company's AI strategy and support your CTO in thinking through their options.

Do you AI?

Before you can effectively advise your executive leadership team on AI adoption, you need to make sure that (a) you know what you're talking about and (b) that they *believe* that you know what you're talking about!

Building Competence

If you want to lead the conversation on AI, you need hands on personal experience - this is a transition that you can't just "manage" through.

The best way to start building intuitions for how and where to leverage LLMs is to use them regularly. You probably already have a daily driver for chat from one of the major labs (OpenAI, Anthropic, Google, Microsoft, etc.). And if you still use an IDE sometimes, hopefully you've got a preferred augmented engineering setup using something like Cursor to lend a hand on your coding projects. Make sure to add an agentic tool like Claude Code, OpenAI's Codex, Google's Antigravity or Factory's droids, to get a sense of both their power and the guard rails required to keep them on track.

Build a research agent

You should look for opportunities to leverage AI within your everyday work. One simple opportunity would be to build your own research agent. That provides the double benefits of both getting hands-on experience with LLMs and providing you with regular updates on how AI is being adopted within your industry and at other engineering orgs.

Whether you choose to build or buy it, you should have a research agent searching and storing articles on how GenAI is being leveraged by your competitors, more broadly across your industry and generally by software engineering orgs. You should

configure it to put together a short weekly briefing that you can edit and share with the executive team-giving them an overview of key GenAI announcements that week along with your synthesis/interpretation as it relates to both AI adoption within your org and trends potentially affecting your competitive landscape. Check out the detailed guidance on this in Part 3 in Chapter 20 on “Building Personal Competence” if you’d like a recipe to follow.

By showing that you’re reading and synthesizing key articles on AI adoption, your executive peers are less likely to keep sending you what they find. And if the rest of the team does have resources, provide them with an email address to send them to so they can be retrieved and summarized by your agent.

Drive the process

However the governance has been set up, as the CTO/SVP engineer, you are responsible for ensuring that your AI plan is appropriate for the business you’re in and the resources you have. You need to make sure that the company goes through a thorough assessment of the potential impact of AI across the organization. You’ll be collaborating with the rest of the C-suite on much of this, but be ready to drive the process if they’re not prioritizing the development of a thoughtful, comprehensive and consistent strategic plan for engaging with AI.

Reviewing your strategy

By now, I’m sure that your company has an AI strategy. But as I speak to engineering leaders, I’m hearing that many companies have policies that are driven by the need to “do something” and influenced by the loudest voices rather than the most thoughtful analysis.

Here’s a top-down way to think about reviewing your AI strategy: First, you need an impact assessment to understand how exposed your company and industry are to AI-driven changes. Then you need to think through your varied stakeholders to understand how they are likely to respond to different strategies. With that context, you then need to make the hard decision where you want to sit on the AI Technology Adoption Lifecycle curve.

Only once you’ve aligned on that can you then make sure that your policies, practices, and incentives align with the real rate at which your organization is willing and able to adopt AI.

Impact Assessment

How much could AI impact your business? This might end up being the biggest driver of how aggressively you need to adopt AI. If the business impact will be mini-

mal, you have the option of taking your time, but if it could be an existential threat, you should make sure to be on the bleeding edge of the adoption curve.

At one extreme, if the majority of the value of your business comes from physical assets that are unlikely to be displaced by technology, you have quite a lot of flexibility. If you own a chain of gyms or ski resorts, those are unlikely to be replaced by AI. You might lose potential operational efficiencies by not leveraging LLMs, but you're unlikely to be put out of business next year.

At the other extreme, if you sell SaaS software or tools for building websites, you need to be very concerned. It's possible that your category could be disrupted by AI first offerings, so you're going to want to be much closer to the front of the adoption curve so you can deliver features and operational efficiencies quickly enough to keep the loyalty of your existing customers.

For example, if you provide productivity suites to corporations, it's not guaranteed that you'll continue to "own the interface" with the employees. Soon they will have agents doing and displaying their work, so there is a risk that you'll be reduced to a supported file format and lose the licensing revenue for many agent-augmented workers.

Equally, if you provide general-purpose, configurable tools that can be coerced into doing a good enough job for most companies (I'm looking at you, Salesforce, Jira, Hubspot, and many others), how many of your customers will put up with clunky interfaces and limits on agentic access to their data in your silos? Many AI-forward companies are already rethinking and rebuilding their own business flows from first principles, using relational databases and deterministic workflows with agentic steps.

There will always be value in lead enrichment or jurisdiction-specific tax rules for W2 employees—but it's by no means certain whether people will want to pay for your interface or whether they'll just want to have API access to the hard parts of the business logic to enhance their homegrown business-process engines. And the risk with the API business is that the *innovators' dilemma* (protecting revenue from current lines of business at the expense of pursuing disruptive opportunities) will make it very hard for you to be the company that moves from high-revenue, high-margin SaaS sales to providing the inexpensive APIs that solves for the same core business complexities.

You also have to take the time to do some scenario planning to think through potential second order effects. If you're in AdTech, what is going to happen when agents replace humans as the primary purchasers online? If you provide news or analysis, what will happen when people stop searching web pages and reading emails and just ask their agents for the outcomes they desire? The good news is that raw information is still going to be required for decision making, and there will probably be a huge spike in the number of real-time auctions when agents are making purchases. But

there is no guarantee that the current incumbents will continue to thrive in these new markets.

It's really important to invest properly in this process. Try to schedule a two- or three-day leadership offsite and spend the first day with a trained facilitator red-teaming your business and your industry, coming up with crazy possible futures and risks to your business model that might not be immediately apparent. When you come to the planning phase, you will have to balance short-term priorities with longer-term investments, but it's important to explore the range of possible futures and to explicitly decide which risks to mitigate and which ones are worth just living with.

Understand your audiences

Every message that your executive team shares is going to be scrutinized by a wide range of stakeholders, so before you pick your adoption strategy, you should think about all of the audiences it might affect and its likely second-order effects.

Customers and Prospects

Again, the impact of your GenAI strategy on your customers will depend greatly on the kind of business you're in. If you manufacture tractors, they probably don't care too much. Equally, if you run a chain of restaurants, LLM adoption probably isn't their first concern.

If you sell software to individuals, they're going to keep an eye on your roadmap. Are you staying competitive with the AI-native competition, or should customers try one of the new AI-first apps that might make their lives easier?

If you're selling software to companies, they definitely don't want to change vendors. The bigger they are, the less they'll want to do it. That said, IT departments are going to be reviewing their licensing and SaaS spend and also investigating operational efficiencies. You have a window of time to prove that you can help them to get the same value out of their existing systems with your agentic offerings that they could get from the new AI-first startups. Announcements and roadmap commitments can be a great way to get them to "hold the line" while you update your systems to help them to deliver the new capabilities that they'll require.

In the B2B software space, prospects will be looking particularly hard at your roadmap. They'll also be considering signaling, like limiting API access to data held within your systems. If I am evaluating a new vendor and they're going to lock me into using their agentic systems, which they haven't shipped yet, I'm going to look a lot more carefully at other solutions that might give me more options with my data down the road.

Employees and New Hires

Some employees are excited to use the latest AI tools. That mindset, and the experimentation that comes from it means that they're likely to become AI-augmented much more quickly and to deliver more business value—at least until the ability to collaborate effectively with AI stops being a differentiator. (Some readers will remember when the ability to use computers set you apart as a potential hire!)

Others are much more cautious or fearful about AI and the potential for disruption. They're less likely to adopt GenAI tooling and more likely to focus on its downsides or limits, rather than pushing through and learning the new ways of working required to collaborate effectively with AI.

Which group do you want to retain? The faster you push on AI adoption, the more likely you are to retain the employees who are excited to leverage AI in their roles. The more conservative you are with adoption, the more likely you are to lose them to other companies that will provide them with opportunities to use the latest tools and time to experiment.

Your messaging on AI adoption and the way you handle hiring will also determine how AI-forward your new hires are likely to be. For example, if you still consider using an LLM to be “cheating” in an engineering interview, you're unlikely to attract the best augmented engineers, who wouldn't consider coding without an AI copilot or an agentic team supporting them.

Investors

If your company is publicly traded, you are only ever as valuable as your last quarterly earnings call, so it's hard to invest in strategies that will pay off in a year or two. But if you can take the pain in terms of option values, the cost of capital, and the potential risk to your continued employment, well-placed bets on AI will pay off handsomely over time. I expect a lot of the incumbents who are focused entirely on making the next quarter will eventually be overwhelmed by competitors who are either AI native or willing to invest more aggressively in AI adoption. The only greater risk to your job than investing too much in AI may be not investing enough.

If you have venture or private equity funding and you're not currently raising or selling, you will probably get a more supportive response from your investors to slightly longer payback periods for your GenAI investments, but they're still going to want to see a thoughtful, ROI-driven approach to mitigate risk and drive aggressive expansion.

Assess your engineering maturity

It's important to set realistic expectations about the current maturity of your engineering org and the impact that will have on your ability to leverage AI. If you still rely on manual QA and deployments and don't score well on developer efficiency metrics like DORA, SPACE, or DX Core 4, it doesn't matter how quickly AI can generate new features—the bottlenecks will develop in reviewing, QAing, and deploying them.

There is a direct correlation between the maturity of your engineering processes and practices and your ability to get value from AI within the SDLC. I'll dig into this much deeper in part 3, but it's important to realize that if you don't already have an efficient, modern SDLC, you'll have to fix that before you can really accelerate software development using AI. The only good news is that GenAI will be able to help you accelerate the process of modernizing your engineering org.

Picking a Lane

As a company, the most important strategic decision you have to make is how aggressively you should invest in adopting AI. AI adoption is not free: it will cost money, time, energy, and focus. Done right, it will provide a substantial return, but it's important to be realistic about the scope of the investment your company is willing to make and honest about the rate at which your company will be capable of adopting AI.

Should your company be an innovator, early adopter, early or late majority, or laggard in terms of adopting AI solutions? There are good reasons to pick many of those strategies, but the most important thing is for your executive team to discuss, align on, and then to pick a lane. Then, as an organization, you need to understand the implications of the choice, own them, and support them appropriately.

The one thing you must do is *be consistent*. If your CEO is trumpeting your investments in AI and your AI-forward stance while your CISO and audit, risk, and legal teams are blocking or slow-rolling all of your proposed AI experiments, you will end up making nobody happy—and the inconsistencies will show up pretty quickly in the results your org is delivering.

Let's look at some illustrative potential pros, cons, and implications for each option and why you might select it.

Innovator

Pros

You'll be one of the first companies to get the benefits of AI accelerated software development. You'll attract and retain the most curious, innovative engineers that really want to push the limits of what AI can do. You'll be able to substantially out

deliver and out innovate most traditional software companies - allowing you to experiment and take advantage of new opportunities quicker than most other businesses.

Cons

You're going to lose anyone on your team that isn't all in on AI. You'll burn a lot of money on experiments that don't work. Your delivery cadence may slow down before it speeds up. You'll end up building and rebuilding a lot of things that later adopters will just be able to license.

Implications

If you want to be all in on AI, you should be focused on accelerating experimentation. You shouldn't be starting with requiring ROI, but simply by identifying and tackling impactful opportunities. You should make tool review/approval for pilots very quick and simple with only the smallest number of essential red lines for tools you won't trial, and you should be open to building your own solutions where there isn't yet an OSS or commercial offering for the problem you're trying to solve. You should require the use of AI in interviewing and be prepared to rapidly restructure your org and your SDLC to support accelerated delivery of business value.

Why choose?

If you already have a modern, high-performing engineering org excited about adopting AI and see either existential threats or exceptional business opportunities, it's worth paying the "innovator tax" of having to build many of the tools that you need to allow you to deliver novel capabilities and capture market share before your competitors can get there.

Early Adopter

Pros

You're going to be able to out-deliver and out-innovate most of your competitors that are less aggressive with AI adoption and attract engineers that are excited and interested in leveraging AI. You'll be able to license tools to accelerate your learning and to learn from the mistakes made by the innovators ahead of you.

Cons

You're still going to be adopting tools before they're proven, so you will need to replace the elements of your stack frequently as other tools or approaches end up proving more popular. You will be able to innovate and deliver quickly, but you probably won't be able to keep up with the innovators and AI native players as they race to deliver blue-ocean solutions to problems that didn't exist last year.

Implications

As an early adopter, you'll still need a transparent, expedited process for approving experiments—and to understand that you won't always be able to prove ROI before you run an experiment. You'll need to be thoughtful about managing your dependencies on specific tools. Have a lightweight end-of-life plan for all of the tools you adopt *before* you start using them.

Why choose?

This is a better strategy if you are in a category that's ripe for disruption, but you have enough stickiness and good will with your customers that they're willing to wait just a little longer for you to deliver on your AI roadmap on top of your existing offerings. Pulling this off successfully will still require you to have, or quickly build, a modern, fairly high-performing engineering org with people who are excited about adopting AI.

Early Majority

Pros

You will get to leverage the proven patterns and tooling that innovators and early adopters trialed. You may still occasionally need to replace a tool or change a process, but generally you'll be able to share proven good practices across the org and move deliberately to roll out changes without freaking out your team. You'll also be able to retain much organizational wisdom and support your existing engineers in adopting the new practices as they solidify.

Cons

You won't be able to keep up with the pace of innovation or delivery of the innovators and the early adopters, so you need to have a lot of customer good will, deep data, or contractual moats—or be in an industry that's less likely to be transformed by AI.

Implications

You should have a council that selects approved tools only when they've proven to be of value and are starting to emerge as the leading players within their segment. Minimize ad-hoc experimentation and focus on improving fundamental processes and tooling within your engineering org, so you'll be ready to accelerate software delivery as you start to adopt proven tools.

Why choose?

This is a good strategy in cases where the impact of AI on your offering will be less pronounced or longer term. It also gives you a little more time to improve the fundamentals of your engineering org if you don't yet have great CI/CD/automation.

Late Majority/Laggards

Pros

You can just keep writing software the way you always have. Your team can stay focused on delivering against your roadmap, and you won't need to attract the best AI talent. With a little training, you'll already have the team you need, with all the business context to keep shipping against your roadmap - eventually accelerating your delivery as the tools you already use become more capable.

Cons

You won't see much of an improvement from AI in your software delivery. You definitely won't be able to keep up with AI natives, innovators, or early adopters.

Implications

You'll probably wonder what all the fuss is about. Eventually, as your current vendors figure out how to leverage AI effectively within their products, you'll see some modest productivity gains. You should avoid experimentation, instead waiting for your current vendors to deliver value to minimize the change required across your org.

Why choose?

If you see little opportunity or threat to your industry from AI, this allows you to keep a laser focus on your business and your existing roadmap. It allows you to keep the same team and doesn't really require much extra from your engineering org. This lane supports continuous, stable development and modest attrition.

There really is nothing wrong with picking any of the lanes I've outlined above. The only issues arise if you pick a lane that's inconsistent with your risk exposure to AI adoption or are inconsistent in your planning and messaging. A SaaS-based website builder probably doesn't want to be a late-majority adopter. And if you communicate that you're an innovator in AI, make sure to back that up with low-friction access to the latest tools and support for learning how to make the most of them, providing whatever time and resources are required to deliver the applications you need.

If you'd like another tool for supporting your strategic planning, I'd strongly recommend checking out Will Larson's book *Crafting Engineering Strategy*. You can even buy a version that you can feed into an LLM, so it can assist you in creating and thinking through your strategic approach!

Once you commit to a lane, the next step is to review your policies and processes to ensure that you're allowing your team to do the work that you're promising to your stakeholders, which I'll discuss in Chapter 4.

What's Your Plan?

A Note for Early Release Readers

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

Once you've picked a lane, you need to ensure that your plan is consistent with your positioning. In this chapter, I start by outlining the key stakeholders to align with and then propose some thoughts for thinking about delivering practical plans for tool approval, hiring, setting the tone and change management/enablement whether you're planning to be an innovator, early adopter, or an early or late majority adopter of AI in the SDLC.

Aligning Across the Org

In addition to picking a lane, it's also important to align tactically with your stakeholders to ensure that your strategy is supported by your tactical plan. Key stakeholders include:

The board

They need to understand your positioning and the impact that it will have on all aspects of your business - whether that's the technology risks of being an innova-

tor on the bleeding edge of AI adoption or the competitive risks of choosing a late majority/laggard positioning.

The CEO

For CEOs, it's hard not to talk a big game on AI adoption, especially when your competitors are breathlessly announcing all of their new AI initiatives. But it's critical that the CEO's messaging is consistent with your strategy, your risk appetite, and the maturity of your engineering org. There is nothing worse than saying you're a leader in AI and then having the CISO, compliance, or legal teams lock down access to the tools required to deliver on that mission.

Your "risk team"

Depending on your company, there are some combination of teams, such as legal, compliance and/or audit groups, whose primary job is to stop you from doing something dangerous. Unfortunately, experimentation is inherently risky. It's important to engage with your risk team about the implications of your AI adoption strategy.

If you're planning on being late majority/laggards, it'll be a pretty easy conversation. If you want to be innovators or early adopters, you're going to need to have hard conversations about minimal red lines and guaranteed approval times for new tools that are measured in days, not weeks.

Your talent team

Your adoption strategy for AI is going to materially impact your recruiting process. For example, if you want to be an innovator, you'll need to retool your hiring rubrics to attract and filter for AI augmented engineers.

Your Engineering org

It's important to lay out realistic expectations for and with your engineering org. If you require that no human read or write code within a quarter, that's a very different mission from suggesting that people "play with AI a bit" while waiting for the tooling from your favored vendor to evolve.

Planning Implications by Adoption Strategy

Let's dig a little deeper into the implications of each adoption strategy on your planning, processes, and messaging.

Innovator

If you want to be on the bleeding edge to capture the potential upside and/or protect against material business disruption risks, you're going to have to push hard on AI

adoption and accept some risks and uncertainty as the cost of moving fast and (potentially) breaking some things.

Tool Approval

Your approval process needs to be measured in days, not weeks. If your risk team can't turn around a review in 3 to 5 business days, that's going to be the bottleneck that determines your actual pace of adoption, regardless of what your strategy says.

Red lines should be minimal and well-defined: a short, explicit list of things you won't do (such as allowing third-party models to train on your data). Everything else is fair game for piloting.

Low-cost experiments (under some agreed threshold) shouldn't require ROI justification upfront. You're paying for learning velocity, not proven returns on every tool.

Expect to be running multiple tools in the same category simultaneously. Different teams will find different tools effective for different codebases and workflows.

Be prepared to build your own tooling where the market hasn't caught up with your needs. If you're truly on the bleeding edge, commercial offerings may not exist yet for some of your use cases.

Have a lightweight deprecation plan for every tool you adopt. At this pace, you'll be replacing things regularly as the landscape shifts. As such you always have to have a planned off ramp before you start using a tool. How are you going to capture the learnings and the context? What would be plausible approaches to replacing the tool assuming they go out of business, get acquired in a way that doesn't ensure the continuity you need or become irrelevant as the industry moves in another direction?

Hiring

Require candidates to use AI tools during the interview process. This isn't a nice-to-have - it's how you signal what kind of engineering org you are. Your rubrics need to assess how effectively candidates collaborate with LLMs: decomposing problems, providing context, validating output, and iterating on results.

Look for curiosity and adaptability over deep expertise with any single tool. The specific tools will change, but the mindset of working with AI won't. Be honest in job postings that this is an AI-forward org. You want to attract engineers who are excited by that and filter out those who aren't.

You will likely lose some experienced engineers who don't want to work this way. That's a real loss - these are people with deep domain knowledge and institutional context. But if you're committed to this lane, you need to be honest about that trade-off.

Setting the Tone

AI adoption is a mandate, not an invitation. This is how your engineering org works now. Many good programmers do not want to become an engineering manager to a bunch of agents. That is fine, but it's not something you can allow within your org. You need to make it clear that their job is to help to encode what “good software” looks like at your company so agents can build quality software. As your software factory matures, they'll then naturally gravitate towards a smaller platform team focused on building and refining the pipelines, infrastructure and tooling to improve the quality of your agentic code generation or towards the problem domain, writing higher level specifications and validations to run experiments or meet customer needs. They should stop writing code very soon, and understand they probably won't be reading it either in a few months.

Leaders also need to be visibly using AI tools themselves, not just endorsing them. If you are not using a tool like claude code and capturing context to improve the quality of your output while starting to build pipelines to automate management tasks and flows, you're not going to understand how to lead your organization.

Frame AI as the new baseline expectation, similar to how using version control or CI/CD is simply expected. It's not about “adopting AI,” it's about how the team delivers software. Be transparent that short-term productivity may dip as people build new skills. Set expectations with the board and the CEO that there will be an investment period before the acceleration kicks in. Celebrate the engineers who push boundaries, and make their experiments visible across the org.

Change Management and Enablement

In an org of 50 people or more, you need at least one dedicated person whose full-time job is managing the logistics of AI adoption: training, comms, unblocking teams, tracking experiments, and sharing what's working. For most companies, DevEx/Platform is a great place for this person to sit so they can act as a conduit between the team building the software factories and the rest of the org that is tasked with shipping reliable software without reading or writing the code. A background in DevRel or a similar level of communication skills (forward deployed or solution engineer) works well. Someone who has history at the company and is respected by both executives and ICs will be best positioned to lead the charge.

For smaller teams, someone is still going to have to focus on helping the team adopt the new working practices. A little like “test infecting” engineers with TDD, “AI infection” is a good way to think about pairing someone who “gets” agentic development with someone who is still trying to get up to speed. In the smallest teams maybe that person is also heavily involved in building the orchestrator and context systems for automating software development, bringing back what they learn from the feature teams to improve the quality of the automations.

Weekly AI demos and monthly roundups should be standard rituals. You should also add regular office hours (either in person or online) where people can ask questions to learn from engineers who are more comfortable coding agentically. The pace of change requires frequent touchpoints.

Invest in structured training—don't just tell people to “watch some YouTube videos.” Hold lunch-and-learns and paired sessions with strong AI practitioners, and dedicate time for hands-on experimentation.

Build and maintain a shared context repository that includes documentation, prompt libraries, experiment results, and proven patterns. This is how you compound learning across the org instead of having every team rediscover the same things.

Your platform/DevEx team should be driving the rollout of tools and practices to feature teams, treating AI enablement as a first-class product they're delivering to their internal customers. The job of a platform team is to enable and accelerate the feature teams building the user valued software. For at least the next 6-18 months their most important job will be to help feature teams use software factories to automate the delivery of high value code.

Expect to revise your processes and rituals frequently. What works this month may not work next month as the tools and your team's capabilities evolve.

Early Adopter

As an early adopter, you'll be able to take a little more time to let things settle, focusing on tools and practices that seem to be getting traction, but you are still going to have to push hard on your team to identify, pilot and scale good practices quickly and with the minimum of red tape and bureaucracy.

Tool Approval

You still need a fast approval process, but it can be slightly more structured than what an innovator org would have. A week to review a new tool is reasonable; a month is not.

You're primarily licensing tools rather than building them, so your evaluation criteria should focus on adoption trajectory, integration quality, and the vendor's staying power. As an early adopter you can't wait for a definitive leader to emerge in every category before adopting it. As such you should evaluate their mission and trajectory. Just like you hire a junior engineer for potential, not just for their current skills or track records, you should look for vendors that are thinking about agentic coding in a way that's consistent with your goals. You should also be thoughtful about whether they are just one model release away from irrelevance - are they building a deep, differentiated moat that will continue to make them economically viable even as the models from the foundation labs become more capable?

Require a lightweight end-of-life plan for every tool before you start using it. You're adopting tools before they're fully proven, so you need to be ready to swap them out. And have clear criteria for when a successful pilot becomes a standard tool. The transition from "some teams are trying this" to "this is how we work" needs to be intentional.

Watch what the innovators are doing and learning. You don't need to repeat their most expensive mistakes. Find a small number of high quality blogs from teams solving similar problems to yours and make sure to get a weekly summary to review and share with your platform team.

Hiring

AI proficiency should be a meaningful factor in hiring decisions, though it doesn't need to be the defining one. Allow and encourage LLM use in interviews, but you may not need to require it the way an innovator org would.

Look for engineers who are enthusiastic about AI and have been using it in their work, but also value engineers with strong fundamentals who are willing to learn. Your job postings should make it clear that AI tools are part of your daily workflow. Don't be ambiguous about this - it's a selling point for the candidates you want to attract.

Setting the Tone

Provide strong encouragement to use AI tools, with a clear direction. This doesn't need to be a mandate, but it should come close. Set the expectation that everyone will be using AI tools and building their proficiency. Leadership should be framing AI adoption as a competitive advantage you're actively investing in, not just an experiment.

Be clear that there's organizational support for the learning curve, but also that the direction is not optional. You're heading here; the question is how quickly you can bring everyone along. Share what you're learning from the innovators ahead of you - both their successes and their cautionary tales. It helps your team feel like the approach is informed, not reckless.

Change Management and Enablement

Having a dedicated employee focus on enablement is still important, though it may be a significant percentage of someone's time rather than a full-time role, depending on the organization's size. Regular sharing rituals such as demos and roundups are valuable but can be slightly less frequent than in an innovator org: biweekly rather than weekly might work.

Your training should be more structured and curated than an innovator org's. You're rolling out proven patterns, not asking everyone to figure it out on their own. Identify internal champions who can support their teams in adopting tools and practices that have already proven effective in your org. If you have a mature platform team, they can be responsible for keeping up with good practices, and you'll be moving at a pace that lightweight but structured training materials (blog posts, simple training sessions) will be more viable than at innovators where good practices may be changing weekly.

Document what's working and create playbooks. You're starting to move from "exploration" to "this is how we do things," and that transition needs clear guidance.

Early Majority

In the early majority, you're ceding the benefits from keeping up on the bleeding edge, but in return, you can wait for good practices to emerge. The pace of change is still going to be much faster than your team is used to, but in the early majority it'll be measured in quarters, not weeks or days.

Tool Approval

Create a formal review council or committee that evaluates tools once they've emerged as leading players in their category. You're not experimenting with unproven tools, you're adopting winners.

ROI expectations are reasonable here. You should be able to point to case studies and proven value from earlier adopters before committing to a tool. It's fair to look at token costs and (other than a small budget for experimentation by your platform team) to require a plausible ROI showing how the token burn will accelerate the roadmap, improve quality or reduce headcount.

Standardize on a smaller set of approved tools rather than allowing teams to use whatever they want. Minimize ad-hoc experimentation. The value at this stage comes from disciplined, org-wide rollout of proven, consistent, supportable tools, not from individual exploration.

Your risk team will be much more comfortable with this pace than they would be with the innovator pace. Use that to your advantage. Get them to genuinely buy into your experiments rather than just tolerating them.

Hiring

AI experience is a plus but not a requirement. You're looking for engineers who are open to using AI tools, not necessarily those who've been on the bleeding edge. Your interview process should accommodate AI tools but doesn't need to require them. The signal you're sending is "we're adopting AI thoughtfully," not "we're all-in."

Focus on strong engineering fundamentals. The tools you'll be rolling out are more mature and well-documented, so the learning curve is less steep, but the engineers still need to be strong enough to validate and review AI-generated output.

Be aware that the most AI-forward candidates may be choosing innovator or early adopter orgs over yours. That's OK if you're retaining solid engineers who will adopt tools as you roll them out.

Setting the Tone

Your tone should be supportive and encouraging, but measured. The message is: "We're rolling out proven AI tools to help you work more effectively, and we'll support you in learning them." This is less about urgency and more about steady, confident progress. Avoid both the breathless enthusiasm of the innovators and the passive indifference of the laggards.

Position AI adoption as part of a broader commitment to engineering excellence and modernization. It's one piece of a larger investment, not the whole story. Make sure your CEO's messaging is calibrated to this pace. Nothing undermines this lane faster than an executive team promising innovator-level AI delivery while the org is rolling things out at an early-majority pace.

Change Management and Enablement

Your training can be more programmatic than exploratory. You're teaching people how to use specific approved tools and established workflows, not asking them to experiment broadly. Group training sessions and structured onboarding for new tools will be more effective than individual experimentation.

Focus on getting the fundamentals of your engineering org right—like CI/CD, test coverage, and deployment automation—so you can actually absorb the acceleration that AI tools provide.

Internal champions are still important, but their role is more about driving adoption of approved tools than about discovering new approaches. Communication cadence can be monthly rather than weekly. The pace of change is more manageable, and you don't want to create artificial urgency.

Late Majority/Laggard

If you've genuinely assessed your risk exposure and concluded that this is the right lane for your business, here's how to be intentional about it rather than just defaulting to inaction.

Tool Approval

Your existing vendor relationships are your primary channel for AI adoption. Wait for your current tools (such as IDEs, CI/CD platforms, and cloud providers) to integrate AI capabilities natively.

A standard procurement process with standard timelines is fine. There's no urgency to evaluate new tools quickly.

Avoid approving niche AI-specific tools. The overhead of managing additional vendors and integrations isn't worth it when you can wait for your established vendors to catch up—and they will, eventually. They're even more motivated than you are, because their early-adopter customers are asking hard questions.

Keep your red lines simple and conservative. You have the luxury of time, so use it to avoid unnecessary risk.

Hiring

AI experience is not a meaningful part of your hiring rubrics. You're hiring for the same skills you always have, with the same interview process.

Don't penalize candidates for using AI in interviews, but don't specifically look for it either.

Recognize that you probably won't attract the most AI-forward engineers. You'll retain people who value stability and are comfortable with established practices, which is consistent with this lane.

As AI capabilities become embedded in your standard tooling, provide enough training that your existing team can take advantage of the new features.

Setting the Tone

AI is available for those who want to use it, but it's not a priority or an expectation. The focus remains on your existing roadmap and your established ways of working.

Avoid dismissing AI as not worthwhile; that alienates the curious members of your team. Instead, frame it as something you're monitoring and will adopt as it matures within the tools you already use.

Be honest with yourself and your team about the trade-offs. If your industry becomes more AI-disrupted than you expected, you'll need to accelerate, and your team should understand that this lane isn't permanent if conditions change.

Make sure the CEO and the board are genuinely aligned with this pace. If they're publicly claiming AI leadership while you're operating as a laggard org, the inconsistency will damage the org's credibility internally and externally.

Change Management and Enablement

Formal AI-specific change management isn't necessary. As your vendors roll out AI features, incorporate them into your standard training and onboarding processes.

No dedicated AI enablement staff are needed. Your platform team can handle the incremental updates as they come. This is one of the genuine advantages of this lane - you're not staffing up an initiative that may need to be restructured every few months.

Keep an eye on what the earlier adopters are learning, so you have context when your vendors start shipping AI features and you need to evaluate them.

Your biggest risk isn't necessarily the lost velocity from slower AI adoption—it's that your most curious and ambitious engineers might leave for orgs that give them more room to experiment. Monitor your attrition patterns for early warning signs.

Part 1 of this book set the stage: You've seen what good and bad look like, you understand that AI amplifies what's already there, you've picked a lane for how aggressively your org should adopt AI, and you've started to align your stakeholders around a plan that's consistent with that strategy.

In Part 2, we're going to get much more specific. What does an AI-powered engineering org actually look like, from how you specify software all the way through to how you deploy, manage, and staff it? We'll walk through each phase of the software development lifecycle - specification, agentic and augmented engineering, code review, testing, deployment, team management, project management, hiring, and onboarding - and look at how AI is already changing the work and where it's headed. The goal is to give you a concrete picture of the destination, so that when we get to Part 3 and start building a roadmap, you'll know exactly what you're building toward.

About the Author

Peter Bell has been building CTO communities for 15 years. He hosts the O'Reilly CTO Hour, facilitates the executive summits for CNCF at KubeCon, and is the founder and CTO of Gather.dev, the global learning community for senior engineering leaders. He is also the founder and CTO of Dev Tool Collective, a community of technical founders of dev, data, and platform tool companies. He has spent most of the last 15 years helping engineering leaders to share and adopt good practices and bridging the gap between new developer tooling and the expectations of the executives looking to adopt them.